KSFUPRO1KU **IT University, F2026**

Written exam, Functional Programming
Friday May 22, 2026

Version 1.00 of May 21, 2026

These exam questions comprise 7 pages. Check immediately that you have all the pages.

The exam duration is 4 hours.

There are 4 questions. To obtain full marks you must answer all the subquestions satisfactorily.

You do not have general Internet access at the exam. You can only access WISEflow. You are allowed to use books, lecture notes, lecture slides, hand ins, solutions to assignments, calculators, computers, and software used throughout the course including your IDE of choice. **You are not allowed to use software utilizing AI, that is, any form of AI dialogue prompt, whether webbased or installed locally on your computer.**

You are allowed, unless otherwise stated, to use the .NET library including the modules described in the book, e.g., `List`, `Set` and `Map`.

If a subquestion requires you to define a particular function, then you may **use that function in subsequent subquestions**, even if you have not managed to define it yourself.

If a subquestion requires you to define a particular function, then you may **define as many helper functions as you want**. In any case, you must define the required function so that it has the same or a more generic type and effect than the subquestion asks for.

The grading will favour functional solutions, i.e., solutions without side effects. Recursion is also favoured over loops. An imperative solution is preferred over no solution.

You should hand in one file only, e.g., `may2026.fsx`. Do not use time on formatting your solution in Word or PDF.

You are welcome to use the accompanying file `may2026Snippets.fsx`. The file contains some of the code snippets included in the exam set for your convenience.

You must include explanations to support your solutions. You must demonstrate your solutions work by running the examples provided.

Your exam hand in must be made by yourself and yourself only, and this holds for program code, examples, the explanations you provide for the code, and all other parts of the answers. It is illegal to make the exam answers as group work or to enlist the help of others in any way. This includes copying solutions from other resources and utilizing help from AI based tools.

Your hand in must contain the following declaration:

I hereby declare that I myself have created this exam hand in in its entirety without help from AI tooling and anybody else.

Question 1 (25%)

Question 1.1

The concept of lists is described in HR Section 5.1. You are **not** allowed to use the `List` library in your answer to these questions:

- Declare an F# function `take` (n, xs) of type `int * 'a list -> 'a list` that evaluates to the list containing the first n elements of list *xs*. The ordering of elements must be preserved. In case n is larger than the number of elements in *xs*, the error message "take: not enough elements." must be raised using `failwith`. For instance, `take (3,[1;2;4;5])` evaluates to `[1;2;4]`.
- Declare an F# function `partition` *p xs* of type `('a -> bool) -> 'a list -> 'a list * 'a list` that partitions the list *xs* into two lists where elements x are in the first list if $p\ x$ is true and otherwise in the second list. For instance `partition ((<)5) [1;2;5;3;5;9;9;10]` may evaluate to `([10;9;9],[5;3;5;2;1])`.
- Tail recursive functions are covered in HR Section 9.4. Declare a tail recursive F# function `partitionA` with same type and semantics as `partition` above. You may reuse `partition` in case `partition` is already tail recursive.
- Declare an F# function `isSorted` *xs* of type `'a list -> bool when 'a: comparison` that evaluates to `true` if the elements in *xs* are sorted; otherwise evaluates to `false`. For instance `isSorted [1;2;3]` evaluates to `true` and `isSorted [2;1]` evaluates to `false`.

  Explain why the type has the additional condition: `when 'a: comparison`.

Question 1.2

The concept of sequences is described in HR Chapter 11. The *Tribonacci* sequence T_n is defined as $T_n = T_{n-1} + T_{n-2} + T_{n-3}$ for $n \geq 3$ and $T_0 = 0, T_1 = 1, T_2 = 1$.

An F# function `tri` implementing the Tribonacci numbers are provided below:

```
let rec tri n =
  if n = 0
    then 0
    else if n = 1 || n = 2
      then 1
      else tri (n - 1) +
           tri (n - 2) +
           tri (n - 3)
```

- Declare an infinite F# sequence `natSeq` of type `int seq` that contains all numbers $n \geq 0$. For instance `Seq.take 4 natSeq` evaluates to `seq [0;1;2;3]`.
- Declare an infinite F# sequence `triSeq` of type `int seq` containing the Tribonacci numbers as defined by the function `tri` above. The sequence must be defined using *sequence expressions*, HR Section 11.6. For instance, `Seq.take 10 triSeq` evaluates to `seq [0;1;1;2;...]`.
- The function `tri` takes a long time to compute for larger n. Declare an F# sequence `triSeq'` of type `int seq` that is a cached version of `triSeq`.

Question 2 (25%)

The concept of finite trees is described in HR Chapter 6. We consider *rose trees* or *n–ary trees* defined with below F# datatype:

```
type RoseTree<'T> =
  | Node of 'T * RoseTree<'T> list
  | Leaf of 'T
type Employee = string * string   // (Name, Title)
```

We will use the rose tree to represent an organization of employees. An employee has a name and title. The F# value `org` below represents an organization:

```
let org =
  Node(("Hanne","CEO" ),
       [ Node(("Bob","CTO"),
              [Node(("Peter","Developer"),[]);
               Node(("Ulla","Developer"),[])]);
         Node(("Kirsten","CFO"),
              [Node(("Hans","Accountant"),[])])])
```

The organization can be drawn as follows:

```
Hanne (CEO)
  |- Bob (CTO)
  |  |- Peter (Developer)
  |  |- Ulla (Developer)
  |- Kirsten (CFO)
     |- Hans (Annountant)
```

Notice, there are several ways of representing the above organization with the `RoseTree<'a>` type.

Question 2.1

Consider another organization `org2` as drawn below

```
Kurt (CEO)
  |- Hanne (CFO)
  |  |- Ulla (Accountant)
  |- Hans (CTO)
     |- Kurt (Manager)
        |- Pia (Developer)
```

- Declare an F# value `org2` of type `RoseTree<string*string>` representing the organization above.
- Declare an F# function *numValues rt* of type `RoseTree<'a> -> int` that evaluates to number of values in the rose tree *rt*. For instance, `numValues org` evaluates to 6.
- Declare an F# function *layers rt* of type `RoseTree<'a> -> int` that evaluates to the number of layers in the rose tree *rt*. For instance, `layers org` evaluates to 3 layers in the organization.

Question 2.2

A rose tree is said to be in *canonical form* if the tree fulfills the property: For all nodes `Node`(v,*rts*) in the rose tree, then *rts* is a non empty list. Any rose tree has a canonical representation because the `Leaf` nodes can be used instead. The tree `org` above is not in canonical form because several nodes, *Node*, have empty lists.

- Declare an F# function `chkRoseTree` *rt* of type `RoseTree<'a> -> bool` that evaluates to `true` if *rt* is in canonical form; otherwise evaluates to `false`. For instance, `chkRoseTree org` evaluates to `false`.

- The rose tree declared below, is the canonical representation of `org`.

```
Node(("Hanne", "CEO"),
     [Node(("Bob", "CTO"),
           [Leaf ("Peter", "Developer"); Leaf ("Ulla", "Developer")]);
      Node (("Kirsten", "CFO"),
            [Leaf ("Hans", "Accountant")])])
```

  Declare an F# function `fixRoseTree` *rt* of type `RoseTree<'a> -> RoseTree<'a>`, that evaluates to the canonical representation of the rose tree *rt*. For instance, `fixRoseTree org` evaluates to the tree shown above.

- It is convenient to know the number of subordinates that each employee in the organization has. Declare an F# function `addNumSubTrees` *rt* of type `RoseTree<'a> -> RoseTree<'a * int>` that evaluates to a new rose tree where all values have been annotated with number of sub trees. For instance, `addNumSubTrees org` evaluates to below rose tree.

```
Node((("Hanne", "CEO"), 5),
     [Node((("Bob", "CTO"), 2),
           [Node ((("Peter", "Developer"), 0), []);
            Node ((("Ulla", "Developer"), 0), [])]);
      Node ((("Kirsten", "CFO"), 1),
            [Node ((("Hans", "Accountant"), 0), [])])])
```

  Employee Hanne has 5 subordinates: Bob, Peter, Ulla, Kirsten and Hans.

Question 3 (30%)

The concept of lists is described in HR Section 5.1. A *zipper* represents a list plus a pointer to a specific focus element, and allows moving the focus left and right. Two example zippers are shown below.

```
1 2 3 4 5 6          'a' 'b' 'c' 'd' 'e' 'f'
    |                 |
```

At left, the zipper contains the numbers `1` to `6`, with the number `3` being the current element that the zipper *focuses* on, illustrated by the bar `|`. At the right, the zipper contains the characters `'a'` to `'f'` and the focus element is character `'a'`. The zipper is represented with the following F# type declaration, (*left*, *focus*, *right*):

```
type 'a Zipper = 'a list * 'a * 'a list // left, focus, right
```

The first list contains elements to the left of the focus element in **reverse** order. The second list element contains elements to the right of focus element in **normal** order. The examples `ex1` and `ex2` represent the two zippers above as two F# values. The element in focus is the second element in the `Zipper` type.

```
let ex1 = ([2;1],3,[4;5;6])
let ex2 = ([],'a',['b';'c';'d';'e';'f'])
```

Your solutions to the tasks below are allowed to have a more general type than the one asked for.

Question 3.1

- Declare two F# values `ex3` and `ex4` representing the two zippers below:

```
"A" "B" "C" "D"      (1,'a') (2,'b') (3,'c')
            |                 |
```

- Explain the types of `ex3` and `ex4` and whether the two types are polymorphic or monomorphic.
- Declare an F# function `numElems` z of type `'a Zipper -> int` that evaluates to number of elements in the zipper z. For instance, `numElems ex1` evaluates to 6.
- Declare an F# function `peekFocus` z of type `'a Zipper -> 'a` that evaluates to the focus element in the zipper z. For instance, `peekFocus ex2` evaluates to `'a'`.
- Declare an F# function `peekRight` z of type `'a Zipper -> 'a option` that evaluates to the element positioned at the right of the element in focus, if exists. In case no element to the right exists, `None` is the result. For instance, `peekRight ex2` evaluates to `Some 'b'`.

Question 3.2

- Declare an F# function `insertAfter` e z of type `'a -> 'a Zipper -> 'a Zipper` that adds element e to the zipper z such that e is to the right of the focus element. The focus element is not changed. Show that `peekRight (insertAfter 42 ex1) = Some 42` evaluates to `true`.
- Declare an F# function `moveLeft` z of type `'a Zipper -> 'a Zipper` that evaluates to a zipper with same order of elements as in z but the focus element is the element at the left of current focus element in z. In case current focus element is leftmost element in z, the result is the same zipper z. For instance, `moveLeft ex1` evaluates to `([1],2,[3;4;5;6])`.

- Declare an F# function `existsLeft` p z of type `('a -> bool) -> 'a Zipper -> bool` that evaluates to `true` if an element e to the left of the focus element in the zipper z exists where `p(`e`)` evaluates to `true`; otherwise evaluates to `false`. For instance, `existsLeft ((=)1) ex1` evaluates to `true`.
- Declare an F# function `moveLeftCircular` z of type `'a Zipper -> 'a Zipper` that moves focus to the element at the left of current focus element similar to `moveLeft`. In case focus element is the leftmost element, the focus element wraps around and becomes the element at the furthest right.

  For instance `moveLeftCircular ex2` evaluates to `(['e';'d';'c';'b';'a'],'f',[])`.

Question 3.3

- Declare an F# function `map` f z of type `('a -> 'b) -> 'a Zipper -> 'b Zipper` that evaluates to a new zipper where the function f has been applied to all elements in z. The new elements preserve the same order as the elements in z. For instance, `map string ex1` evaluates to `(["2";"1"],"3",["4";"5";"6"])`.
- Declare an F# function `foldLeft` f e z of type `('b -> 'a -> 'b) -> 'b -> 'a Zipper -> 'b` that folds the function f over the focus element and the elements to the left. The initial accumulating value is e. In case of `ex1`, with focus element 3, the folding should visit elements in order 3, 2 and 1:

  ```
  1 2 3 4 5 6
   <- |
  ```

  For instance, `foldLeft (+) 0 ex1` evaluates to 6.
- Show the result of evaluating `foldLeft (-) 0 ex1`. Write up the evaluation steps leading to the result.

Question 4 (20%)

Consider the recursive definition of *Delannoy numbers*, *D*, below

$$D(m,n) = \begin{cases} 1 & \text{for } n = 0 \text{ or } m = 0 \\ D(m-1,n) + D(m,n-1) + D(m-1,n-1) & \text{for } n > 0 \text{ and } m > 0 \end{cases}$$

Question 4.1

- Declare an F# function `delannoy` (*m*,*n*) of type `int * int -> int` that implements the Delannoy numbers defined above. For instance, `delannoy (5,5)` evaluates to 1683.
- Explain whether your `delannoy` function is tail recursive or not.

Question 4.2

The implementation of Delannoy numbers below tries to speed up the computation using a mutable dictionary, HR Section 8.11.

```
let delannoyCache (m,n) =
  let c = System.Collections.Generic.Dictionary<int*int,int>()
  let rec delannoy' (m,n) =
    match c.TryGetValue((m,n)) with
      (true, v) -> v
    | _ ->
      if m = 0 || n = 0
        then 1
        else let r1 = delannoy' (m-1,n)
             let r2 = delannoy' (m,n-1)
             let r3 = delannoy' (m-1,n-1)
             let r = r1+r2+r3
             c.Add((m,n),r)
             r
  delannoy' (m,n)
```

- Declare an F# function `delannoyCache2 (`m`,`n`)` of type `int * int -> int` that implements Delannoy numbers. It is required that the function uses an immutable cache, e.g., a Map as explained in HR Section 5.3. For instance, `delannoyCache2 (5,5)` evaluates to 1683.
- Explain why the use of a cache is expected to speed up the computation.

Question 4.3

- Declare an F# function `genMNs` N of type `int -> (int * int) list` that creates this list of $N * N$ pairs:

  $$[(1,1);(1,2);\ldots;(1,N);(2,1);\ldots;(2,N);\ldots;(N,N-1);(N,N)]$$

  For instance, `genMNs 2` evaluates to `[(1,1);(1,2);(2,1);(2,2)]`.
- Declare an F# function `delannoyXS` *xs* of type `(int * int) list -> int list` that computes the Delannoy numbers on the list of pairs *xs*. For instance, `delannoyXS (genMNs 2)` evaluates to `[3;5;5;13]`. You may use any Delannoy function above.